



Ratpack Bot — v1.0 Build Plan

A Discord bot for ToxicRatz. Music, games, outages, and one very opinionated pet rat.

Stack: Node.js 22 · discord.js v14 · Lavalink v4

Host: Docker Compose (self-hosted)

Scope: Multi-guild (1–2 now, more later)

Target: Single builder (Claude), modular

Contents

1. Project Overview & Guiding Principles
2. Tech Stack & Infrastructure
3. Design Language (Global Rules)
4. Module Map & File Structure
5. Database Schema
6. Module 1 — Setup & Configuration
7. Module 2 — Music & Playback
8. Module 3 — Game Data & News
9. Module 4 — Outage Monitoring
10. Module 5 — Peaches the Pet Rat
11. Permissions & Security
12. Environment Variables
13. Build Order & Milestones
14. Risks, Fallbacks & Open Questions

1. Project Overview & Guiding Principles

Ratpack is a self-hosted Discord bot built for **ToxicRatz**. It begins as a music bot but is designed as a modular platform that grows with new features over time. The overall theme is rats and the Rat Pack, but **clarity beats theme** — commands use plain, obvious names.

Core Principles

- **Visual first.** Every response is an embed. No plain-text walls, ever.
- **Buttons over typing.** Anything done more than once gets a button. Anything requiring a name/ID gets autocomplete or a select menu.
- **Modular by default.** Each feature is a self-contained module registering its own commands, listeners, DB tables, and UI. Music is `modules/music`, Peaches is `modules/peaches`, etc. Adding feature #6 never touches feature #1's code.
- **Multi-guild from day one.** Everything keyed by `guild_id`. Zero hardcoded IDs.
- **Ephemeral for personal, public for shared.** Settings and errors are ephemeral. Now-playing, outages, and Peaches are public.

- **Flavor is garnish, never a blocker.** Rat jokes never hide the actual information.
- **Errors state the problem AND the fix.** One line, with a link to the channel or setting that needs attention.
- **No permanent punishments.** Peaches never dies. Playback never deletes history. Mischief never breaks anything.

2. Tech Stack & Infrastructure

Runtime & Libraries

Component	Choice	Why
Runtime	Node.js 22	LTS, native fetch, current discord.js support
Discord library	discord.js v14	Deepest ecosystem for buttons, select menus, modals, scheduled events
Audio engine	Lavalink v4 (Java)	Handles all source platforms; far more resilient to YouTube changes than pure-Node
Lavalink wrapper	lavalink-client	Modern, actively maintained
Lavalink plugins	youtube-source , LavaSrc	YouTube reliability + Spotify/Apple/Deezer/Tidal metadata
Database	SQLite via better-sqlite3	Zero-config, file-based, ideal for self-hosted
ORM	Drizzle ORM	Type-safe, easy Postgres migration later
Scheduler	node-cron + jitter	News polling, outage checks, Peaches spawns
HTTP	Native fetch	No dependency needed
Logging	pino	Fast, structured, JSON-friendly

Docker Compose Layout

```

ratpack/
├── docker-compose.yml
├── .env                # secrets (never committed)
├── data/              # SQLite volume + Lavalink cache
├── bot/
│   ├── Dockerfile
│   └── (Node app)
└── lavalink/
    ├── Dockerfile
    └── application.yml

```



Home-hosting advantage: Residential IPs are far less likely to be flagged by YouTube bot detection than datacenter IPs. This materially de-risks the single biggest failure mode for music bots. Both containers run with `restart: unless-stopped` .

3. Design Language (Global Rules)

These rules apply to *every* module. When in doubt, follow them.

Color Semantics

Color	Hex	Meaning
 Toxic green	#39FF14	Music, success, all-good states
 Amber	#FFB300	Warnings, degraded states, cooldowns
 Red	#E53935	Outages, errors, critical states
 Grey	#4A4A4A	Info, config, neutral

Embed Rules

- Title + description + optional thumbnail. Author field used for "who triggered this."
- Footer always shows the bot name + a rat emoji: 🐹 *Ratpack*.
- Timestamp on any event-based embed (spawns, alerts, news).
- Images: use thumbnails for avatars/album art, full image only for Peaches mischief.

Interaction Rules

- **Ephemeral:** all settings, errors, personal queries, search results.
- **Public:** now-playing, queue, outages, news, Peaches spawns, leaderboards.
- **Buttons:** state changes (pause, skip, mute, feed). Disable when action is unavailable.
- **Select menus:** anything requiring a choice from a list (search results, playlists, channels, services).
- **Autocomplete:** free-text fields that resolve to known values (playlists, services, game names).
- **Cooldowns:** per-user, per-action. Never global unless the action itself is global (e.g. */pet calm*).

Error Format

🐹 I can't post in #rat-nest – I'm missing ****Send Messages****.
Fix it here → [channel link]

Problem, cause, fix. One line, plus link.

Emoji Anchors

🐹 Ratpack · 🎵 music · 🎮 games · 📰 news · 🔔 outages · 🍌 Peaches

4. Module Map & File Structure

```
ratpack/bot/src/
├── index.js                # entry: client login, module loader
├── config.js              # env parsing, constants
├── core/
│   ├── client.js          # discord.js Client with intents
│   ├── commandRegistry.js # slash command registration (global)
│   ├── moduleLoader.js    # auto-loads modules/*/index.js
│   ├── embeds.js          # embed builders (success/warn/error/info)
│   ├── permissions.js     # /setup check, missing-perm reporting
│   ├── logger.js          # pino instance
│   └── scheduler.js       # cron wrapper with jitter
├── db/
│   ├── index.js           # drizzle client, SQLite file
│   ├── schema.js         # all tables (single source of truth)
│   └── migrations/
├── lavalink/
│   ├── manager.js         # lavalink-client init, node config
│   └── events.js          # trackStart, trackEnd, queueEnd, errors
├── modules/
│   ├── setup/             # /setup flow, channel picker, welcome button
│   ├── music/             # /play /skip /queue /pause /volume /playlist
│   │   ├── index.js
│   │   ├── commands/
│   │   ├── ui/panel.js    # The Rat Nest control panel
│   │   ├── ui/search.js   # /sniff results + select menu
│   │   └── sources.js     # source priority, resolution fallbacks
│   ├── games/            # /stats /news /server
│   │   ├── index.js
│   │   ├── providers/
│   │   │   ├── steam.js
│   │   │   ├── fortnite.js
│   │   │   ├── overwatch.js # OverFast API
│   │   │   ├── valorant.js  # valorant-api.com + Riot
│   │   │   ├── dbd.js       # DBD-Database wiki API
│   │   │   └── zomboid.js    # RCON (optional)
│   │   └── newsScheduler.js
│   ├── outages/          # /outage /status
│   │   ├── index.js
│   │   ├── providers/
│   │   │   ├── downstatus.js
│   │   │   ├── officialStatus.js # Epic, Riot, etc.
│   │   │   └── steamServer.js
│   │   ├── checker.js     # 5-min poll with jitter
│   │   └── notifier.js    # dedupe + @here / role ping
│   └── peaches/          # /pet, mischief, leaderboard
│       ├── index.js
│       ├── state.js       # stats, decay, mood calc
│       ├── spawner.js     # normal spawn scheduler
│       ├── mischief.js    # mischief scheduler + escalation
│       ├── ui/embed.js    # Peaches embed builder (mood-aware)
│       └── crossModule.js  # hooks into music/news/outages
└── utils/
    └── format.js          # durations, lists, progress bars
```

```
| fuzzy.js  
| ratpack.js
```

```
# fuzzy matching for autocomplete  
# flavor text pools
```

Module Contract

Every module exports:

```
export default {  
  name: 'music',  
  commands: [...],          // slash command builders  
  buttons: {...},           // button ID → handler  
  selects: {...},           // select menu ID → handler  
  modals: {...},            // modal ID → handler  
  events: [...],            // discord.js events this module cares about  
  cron: [...],              // scheduled tasks  
  init: async (client) => {} // runs on bot ready  
}
```

The loader in `core/moduleLoader.js` wires everything up. A module that is disabled via config simply isn't loaded.

5. Database Schema

All tables include `created_at` / `updated_at` where meaningful. Foreign keys enforce integrity. All guild-scoped tables use `guild_id TEXT NOT NULL`.

Setup & Config

```
guilds(
  id TEXT PRIMARY KEY,
  panel_channel_id TEXT,          -- #rat-nest (music fallback)
  news_channel_id TEXT,          -- #game-updates
  outage_channel_id TEXT,        -- #server-status
  log_channel_id TEXT,           -- #ratpack-logs
  timezone TEXT DEFAULT 'UTC',
  default_mention TEXT,          -- 'here' | 'role' | 'none'
  default_role_id TEXT,
  flavor_enabled INTEGER DEFAULT 1,
  created_at INTEGER
)
```

Music

```
hoards(                                -- user playlists
  id TEXT PRIMARY KEY,
  guild_id TEXT NOT NULL,
  owner_id TEXT NOT NULL,
  name TEXT NOT NULL,
  is_public INTEGER DEFAULT 0,
  created_at INTEGER
)

hoard_tracks(
  id TEXT PRIMARY KEY,
  hoard_id TEXT NOT NULL,
  position INTEGER NOT NULL,
  title TEXT, artist TEXT, uri TEXT,
  duration INTEGER, source TEXT
)

voice_panels(                          -- active panel per VC
  guild_id TEXT,
  voice_channel_id TEXT PRIMARY KEY,
  message_id TEXT,
  updated_at INTEGER
)
```

Games & News

```
game_news_subscriptions(  
  id TEXT PRIMARY KEY,  
  guild_id TEXT NOT NULL,  
  game_name TEXT NOT NULL,  
  steam_appid TEXT,  
  enabled INTEGER DEFAULT 1,  
  tag_filter TEXT,          -- comma-separated Steam news tags  
  last_checked_at INTEGER  
)
```

Outages

```
outage_monitors(  
  id TEXT PRIMARY KEY,  
  guild_id TEXT NOT NULL,  
  service_slug TEXT NOT NULL, -- e.g. 'steam', 'discord'  
  service_name TEXT NOT NULL,  
  enabled INTEGER DEFAULT 1,  
  last_status TEXT,          -- 'up' | 'down' | 'degraded'  
  last_checked_at INTEGER,  
  last_notified_at INTEGER  
)  
  
user_outage_prefs(  
  user_id TEXT,  
  guild_id TEXT,  
  service_slug TEXT,  
  notify_enabled INTEGER DEFAULT 1,  
  PRIMARY KEY (user_id, guild_id, service_slug)  
)
```

Peaches (the pet rat)

```
peaches_state(  
  guild_id TEXT PRIMARY KEY,  
  name TEXT DEFAULT 'Peaches',  
  hunger INTEGER DEFAULT 80,  
  thirst INTEGER DEFAULT 80,  
  happiness INTEGER DEFAULT 80,  
  energy INTEGER DEFAULT 80,  
  cleanliness INTEGER DEFAULT 80,  
  mood TEXT DEFAULT 'content', -- thriving|content|restless|grumpy|neglected  
  level INTEGER DEFAULT 1,  
  last_spawn_at INTEGER,  
  last_decay_at INTEGER,  
  renamed_at INTEGER  
)  
  
peaches_spawns(  
  id TEXT PRIMARY KEY,  
  guild_id TEXT NOT NULL,  
  channel_id TEXT NOT NULL,  
  message_id TEXT,  
  spawned_at INTEGER,  
  expires_at INTEGER,  
  variant TEXT, -- normal|fancy|angry|sneaky|queen  
  resolved INTEGER DEFAULT 0  
)  
  
peaches_interactions(  
  id TEXT PRIMARY KEY,  
  spawn_id TEXT,  
  user_id TEXT,  
  action TEXT, -- feed|water|play|sleep|clean  
  created_at INTEGER  
)  
  
peaches_mischief(  
  id TEXT PRIMARY KEY,  
  guild_id TEXT NOT NULL,  
  channel_id TEXT NOT NULL,  
  message_id TEXT,  
  type TEXT, -- crumbs|gift|turd|hole|shred|walls|ruler  
  spawned_at INTEGER,  
  expires_at INTEGER,  
  resolved INTEGER DEFAULT 0,  
  resolved_by TEXT,  
  resolved_at INTEGER  
)  
  
peaches_stats(  
  guild_id TEXT,  
  user_id TEXT,  
  cheese_fed INTEGER DEFAULT 0,  
  turds_cleaned INTEGER DEFAULT 0,  
  holes_investigated INTEGER DEFAULT 0,  
  walls_knocked INTEGER DEFAULT 0,  
  fealty_pledged INTEGER DEFAULT 0,  
  PRIMARY KEY (guild_id, user_id)  
)
```

```
peaches_config(  
  guild_id TEXT PRIMARY KEY,  
  mischief_enabled INTEGER DEFAULT 1,  
  escalation_enabled INTEGER DEFAULT 1,  
  mischief_frequency TEXT DEFAULT 'normal', -- low|normal|high  
  cross_music INTEGER DEFAULT 1,  
  cross_news INTEGER DEFAULT 1,  
  cross_outages INTEGER DEFAULT 1,  
  cross_hoards INTEGER DEFAULT 1,  
  spawn_channels TEXT          -- comma-separated channel IDs  
)
```

6. Module 1 — Setup & Configuration

Entry Points

- First time the bot joins a guild → posts a welcome embed with a 🗨️ **Set Me Up** button.
- `/setup` always works, even after initial setup.

The Flow

1. **Welcome embed** — brief explainer, one button: *Set Me Up*.
2. **Channel configuration** — for each of the four channels (music fallback, news, outages, logs):
 - Button: 📁 **Pick Existing** → channel select menu filtered to channels the bot can post in
 - Button: 🆕 **Create New** → bot creates a category 🗨️ **Ratpack** with the four channels
3. **Timezone** — select menu with common zones + search
4. **Default notification ping** — buttons: @here · Choose Role · None
5. **Confirm screen** — full summary embed + ✅ **Confirm** · ✎ **Edit**

Re-running Setup

`/setup` shows current config with buttons to change individual pieces. Never forces a full re-run.

Verification

`/setup check` — iterates every configured channel and reports ✅/❌ for: exists, bot can view, bot can send, bot can embed. Any ❌ includes a one-click fix link.

7. Module 2 — Music & Playback

Source Support

Tier 1 — Zero-Setup, Enabled by Default

Source	Method
YouTube / YT Music	<code>youtube-source</code> plugin (best available)
SoundCloud	Lavalink built-in
Bandcamp	Lavalink built-in
Twitch	Lavalink built-in (streams + VODs)
Vimeo	Lavalink built-in
Direct URLs / radio / HLS	Lavalink HTTP source
Discord attachments	File URL → HTTP source

Tier 2 — Optional Credentials (via `.env`)

Source	Credential	Unlocks
Spotify	Client ID + Secret	Reliable metadata, private playlists, liked songs
Apple Music	Token	Apple playlist/album links
Deezer / Tidal	Keys	Higher-fidelity metadata
YouTube	Optional cookies / poToken	Resilience if bot detection tightens

Graceful degradation: if Spotify creds are missing, a Spotify link still resolves — via public metadata → YouTube match. It just says "🕵️ *sniffed this out the hard way.*" Never errors out. `/sources` shows live vs. credential-gated platforms.

Commands

Command	Description
<code>/play <query url></code>	Search or play a link. Autocomplete on known sources.
<code>/search <query></code>	Returns 10 results in a select menu → Play now / Add to queue
<code>/queue</code>	Paginated queue view with total duration
<code>/skip</code>	Vote-skip (threshold configurable) or force-skip with DJ role
<code>/pause</code> / <code>/resume</code>	Toggle playback
<code>/stop</code>	Stop and clear queue
<code>/volume <0-200></code>	Set volume; shows current in embed
<code>/loop <off track queue></code>	Loop mode
<code>/shuffle</code>	Shuffle current queue
<code>/seek <time></code>	Jump to timestamp
<code>/lyrics</code>	Fetch lyrics via LRCLIB
<code>/playlist create add remove list play delete rename save-current import export</code>	Full playlist (hoard) management
<code>/effects <preset></code>	bassboost, nightcore, vaporwave, 8D, karaoke, tremolo
<code>/sources</code>	Show live vs. credential-gated platforms

The Rat Nest Panel (Voice Channel Side Chat)

Posted into the voice channel's text chat when the bot joins. Edit-in-place, never reposted (unless deleted or bot moves VCs).

Panel Layout

- **Now Playing block:** title, artist, thumbnail, duration, requester, source badge, text progress bar
- **Row 1:** ⏮ Previous · ⏸ Pause/Resume · ⏭ Skip · ■ Stop · 🔄 Shuffle
- **Row 2:** 🔁 Loop · ⏮ -10s · ⏭ +10s · 📋 Queue · 🗑 Clear
- **Row 3:** 🔊 Vol - · 🔇 Mute · 🔊 Vol + · Volume select (0–200%)
- **Row 4:** Queue select — jump to track or remove
- **Row 5:** Playlist (hoard) select — load into queue

Behavior Rules

- Buttons disable/hide contextually (no Previous when history is empty).

- Progress bar refreshes max every 10 seconds. All other updates are event-driven.
- Reposts itself if deleted; rebuilds from Lavalink state after bot restart.
- If bot lacks Send permission in the VC text chat, falls back to `#rat-nest` and says so.
- When the bot moves VCs, old panel is deleted and a new one is posted.

Discoverability Bridge

When a song starts, also posts a short auto-deleting notice in the text channel where the command was run: " 🎵 *Now playing in [VC] → [jump to panel]*". Auto-deletes after 30s.

Voice Behavior

- 48kHz stereo Opus, 256kbps when boosted
- Auto-disconnect on empty channel (configurable)
- Auto-pause when alone in channel
- 24/7 mode toggle per guild
- DJ role configurable; vote-skip threshold configurable

8. Module 3 — Game Data & News

Game Stats Providers

Game	Provider	Auth	What It Returns
Fortnite	Epic Fortnite Data API	None	Island metrics, player counts, retention
Overwatch 2	OverFast API	None	Player profiles, hero stats, rank
Valorant	valorant-api.com + Riot API	Riot key (free dev works)	Agents/maps/weapons + player match history + rank
Dead by Daylight	DBD-Database wiki API	None	Killer/Survivor perks, characters
Project Zomboid	RCON (if server present)	Server password	Server status, player list
PEAK	Steam News only	None	Patch notes, updates
Steam	Steam Web API	Free API key	Player summaries, current game, news, server info

DBD caveat: no public player-stats API exists. The wiki API covers perks/characters only. Player stats would require an unofficial client with ToS risk — **skipped by design**.

Commands

Command	Description
<code>/stats <game> <username></code>	Player stats (Fortnite / OW / Valorant)
<code>/server <game></code>	Check if a game's servers are up
<code>/news <game></code>	Latest patch notes / updates
<code>/steam <steamid></code>	Steam profile + current game

Dynamic Steam News System

Users never need to know Steam AppIDs — the bot resolves game names to AppIDs automatically.

Commands

Command	Description
<code>/news add <game></code>	Fuzzy-search Steam catalog → confirm → store AppID
<code>/news add <appid></code>	Direct AppID for power users
<code>/news list</code>	Followed games, with  /  toggle per game
<code>/news remove <game></code>	Stop tracking
<code>/news check</code>	Force refresh now
<code>/news filter <game> <tags></code>	Narrow feed with Steam news tags (e.g. <code>patchnotes</code>)

Default Followed Games at Setup

DBD, Overwatch 2, Project Zomboid, Fortnite, Valorant, PEAK. All removable, all others addable.

Polling

Cron every 6 hours. New posts since last check are posted as embeds in `#game-updates` with title, excerpt, and source link.

9. Module 4 — Outage Monitoring

Provider Chain

1. **DownStatus API** (primary) — free, no key, crowdsourced + official statuses. Supports category queries and 7-day history.
2. **Official status pages** (fallback for services that have them) — Epic Games, Riot Games.
3. **Steam** `ISteamWebAPIUtil/GetServerInfo` — Valve backend health.
4. **Outagescope / Technobezz** — for services without official status APIs (DBD, Overwatch 2).

Default Monitored Services

Category	Services
Games	Dead by Daylight, Overwatch 2, Fortnite, Valorant, Project Zomboid
Platforms	Steam, Discord, Epic Games, Riot Games
Media / Social	Spotify, Twitch, YouTube, X / Twitter, Reddit
General	Cloudflare, AWS, Google, Microsoft 365

~18 services at 5-min interval = ~5,184 requests/day. Well within limits.

Commands

Command	Description
<code>/outage list</code>	Status board of everything monitored
<code>/outage search <query></code>	Search DownStatus catalog (hundreds of services)
<code>/outage add <service></code>	Start monitoring (autocomplete)
<code>/outage remove <service></code>	Stop monitoring
<code>/outage check <service></code>	Manual status check
<code>/outage notify <service> <on/off></code>	Personal ping toggle (per user)
<code>/outage channel <channel></code>	Move alert channel
<code>/status</code>	Overall status board — all monitored services at a glance

Notification Behavior

- Default: `@here` ping on down transition, in `#server-status`
- Per-user opt-out via `/outage notify` — service still visible in channel, just no ping
- Recovery notifications: "🐭 *The rats report Valorant servers have crawled back online.*"
- **No ping spam:** once marked down, bot won't re-ping until recovery + re-down

Polling

Cron every 5 minutes with jitter (not at exact :00).

10. Module 5 — Peaches the Pet Rat

Concept

Peaches is a **server-wide shared pet**. The whole guild collectively keeps her happy. She appears in random opted-in channels, and her mood determines what she does — from wholesome to absolute menace.

Default Name & Rename

- Default name: **Peaches**
- `/pet rename <name>` — rate-limited to once per week per guild
- All flavor text uses the current name dynamically. **No hardcoded "Peaches" anywhere in the code.**
- 1–32 chars, trimmed, no mentions, basic slur filter
- Public announcement embed when renamed: " *Peaches has been renamed to Sir Nibbles by ToxicRatz. The rats have taken note.*"

Normal Spawns

- Cron with randomized interval: default every 4–8 hours (configurable)
- Posted as embed in random opted-in channel
- Only one active spawn at a time per guild
- Previous spawn expires when new one appears

Interaction Buttons

Button	Effect	Cooldown
 Feed Cheese	+happiness, +hunger	30 min per user
 Give Water	+happiness if thirsty	30 min
 Play	+happiness, -energy	1 hour
 Let Her Sleep	Restores energy, blocks other actions briefly	—
 Clean Cage	Required periodically or happiness decays faster	2 hours

First user to click gets credit in flavor text. Multiple users can interact with the same spawn.

Stats & Mood

Hidden stats decay over real time: **hunger, thirst, happiness, energy, cleanliness.**

Mood	Trigger	Display
 Thriving	All stats high	"Peaches is living her best life."
 Content	Default	"Peaches is doing fine."
 Restless	One stat low	"Peaches keeps staring at the cheese drawer."
 Grumpy	Two+ stats low	"Peaches has knocked something off a shelf. Deliberately."
 Neglected	Critically low 48h+	"Peaches has gone feral. Someone should fix this."

No permanent punishment. She never dies — just gets grumpier.

Mischief System

Separate scheduler, probability weighted by mood. Runs only in opted-in channels, respects global cooldown.

Mood	Mischief	Flavor
 Thriving	 Cheese crumbs dropped	"Peaches left you a snack. You're welcome."
 Content	 Tiny gift (random emoji reaction)	"Peaches has blessed this message."
 Restless	 Rat turd with  Clean Up button	"Peaches has been here. Evidence attached."
 Grumpy	 Hole image	"She found a hole. She is thinking about it."
 Neglected	 Escalation (see below)	"Peaches has gone feral."

Turd Mechanic Details

- Posted in random eligible channel
-  **Clean Up** button — first click removes, increments turds_cleaned counter
- If not cleaned within 30 min, edits in place: "This has been here a while. Peaches is pleased."
- Still cleanable after edit — just sassier

Hole Mechanic Details

- Buttons:  **Investigate** and  **Ignore Her**
- Investigate → "You peer into the hole. Peaches stares back. Nothing happens. You feel watched." Small chance of joke reward.
- Ignore → next mischief has slightly higher escalation odds

Escalation Ladder (Neglected, 48h+)

1.  Extra holes across multiple channels
2.  "Peaches has shredded the toilet paper" — fake **Restore Order** button
3.  "Peaches has moved into the walls" — poll: **Knock on the wall** / **Ignore it**
4.  "Peaches has declared herself ruler of this server" — **Pledge Fealty** resets mood instantly (escape hatch)

Recovery is always one click.

Cross-Module Mischief

- **Music:** occasionally adds a track to queue with requester "*Peaches* 🍑"
- **News:** replies to a Steam news post with "*Peaches has also seen this. She is unimpressed.*"
- **Outages:** on recovery, occasional comment: "*The rats have restored order.*"
- **Hoards:** auto-generated "**Peaches' Hoard**" playlist she "curates"
- **Emoji Thief:** if neglected, she "steals" a random emoji — flavor text only, emoji still works

All cross-module mischief:

- Toggleable per module (`/pet mischief music off`)
- Rate-limited
- Silent if the target module is disabled in that guild

Commands

Command	Description
<code>/pet</code>	Status embed with stats + current mood
<code>/pet stats</code>	Per-user stats (cheese fed, turds cleaned, etc.)
<code>/pet leaderboard</code>	Weekly rankings, multiple categories
<code>/pet channels</code>	Choose where Peaches can appear
<code>/pet config</code>	Spawn frequency, mischief frequency, escalation on/off
<code>/pet mischief</code>	Toggle which mischief types and cross-module hooks are allowed
<code>/pet rename</code> <code><name></code>	Rename Peaches (once per week)
<code>/pet calm</code>	Emergency reset — mood → Content, clear pending mischief. 1h cooldown, any member.

Design Guardrails

- No pings for mischief. Ever.
- Max one active mischief per channel
- Auto-expire unresolved mischief after 1–2 hours
- Fully silent when module disabled
- **Never deletes real messages. Never modifies other users' content.** All mischief is bot-authored embeds only. Safe with admin perms.
- Tone: mischievous, never mean. Cut any joke that could land as an insult toward a specific user.

11. Permissions & Security

Recommended Grant

Administrator — for a self-hosted bot in her own servers, this collapses every permission headache into one checkbox and lets `/setup` auto-create channels without debugging overwrites.

Tradeoff: Administrator is all-or-nothing. If the token leaks, the bot can do anything in any guild it's in. Grant admin, but respect the three guardrails below.

Guardrails

1. **2FA enabled** on the bot's Discord account. Non-negotiable with admin perms.
2. **Token stays in `.env` only** — never committed, never pasted into chat. Rotate immediately if leaked.
3. **Curated permission set documented below** so if the bot ever goes to public servers, you can swap in ~11 specific perms instead.

Curated Permission Set (for future)

View Channels • Send Messages • Embed Links • Attach Files • Read Message History • Add Reactions • Use External Emojis • Manage Messages • Manage Channels • Connect • Speak • Mention Everyone

Gotchas

- `@here` pings require **Mention Everyone** — needed for outage alerts.
- VC side chat panel needs **Send Messages on that specific voice channel**, not just the server. Admin bypasses this.
- Auto-creating the 📁 Ratpack category needs **Manage Channels**.

12. Environment Variables

```
# Core
DISCORD_TOKEN=
DISCORD_CLIENT_ID=
DISCORD_GUILD_ID_DEV=          # optional: fast command reg in dev

# Database
DATABASE_PATH=./data/ratpack.db

# Lavalink
LAVALINK_HOST=lavalink
LAVALINK_PORT=2333
LAVALINK_PASSWORD=

# Optional: Spotify
SPOTIFY_CLIENT_ID=
SPOTIFY_CLIENT_SECRET=

# Optional: Apple Music
APPLE_MUSIC_TOKEN=

# Optional: Deezer / Tidal
DEEZER_KEY=
TIDAL_KEY=

# Optional: YouTube resilience
YOUTUBE_COOKIE=
YOUTUBE_PO_TOKEN=

# Optional: Steam
STEAM_API_KEY=                 # free from steamcommunity.com/dev/apikey

# Optional: Riot (for Valorant match history)
RIOT_API_KEY=

# Optional: Project Zomboid RCON
ZOMBOID_RCON_HOST=
ZOMBOID_RCON_PORT=
ZOMBOID_RCON_PASSWORD=

# Behavior
LOG_LEVEL=info
FLAVOR_ENABLED=true
```

13. Build Order & Milestones

#	Deliverable	Depends On
M0	Repo scaffold, Docker Compose, bot login, global slash command registration, module loader, DB migrations runner	—
M1	Lavalink container running, <code>/play <url></code> joins VC and plays	M0
M2	<code>/search</code> + select menu, <code>/queue</code> , <code>/skip</code> , <code>/stop</code> , core music commands	M1
M3	The Rat Nest voice-channel panel with full controls (buttons, selects, state, restart recovery)	M2
M4	Playlists (hoards): CRUD, play, save-current, import/export	M3
M5	Setup module: welcome button, channel picker, auto-create, verification	M0
M6	Game stats providers (Fortnite, OW, Valorant, DBD, Steam) + <code>/stats</code> , <code>/steam</code>	M5
M7	Steam news system: dynamic add, fuzzy search, polling scheduler	M5
M8	Outage monitoring: DownStatus + fallbacks, poll scheduler, per-user notify prefs, <code>/status</code>	M5
M9	Peaches v1: spawns, interactions, stat decay, mood, <code>/pet</code> commands	M5
M10	Peaches mischief system: turds, holes, escalation ladder, cleanup buttons	M9
M11	Cross-module Peaches hooks (music queue, news replies, outage recovery comments)	M10
M12	Audio effects (Sewer Effects), lyrics, vote-skip, 24/7, polish pass	M4
M13	Dockerize for production, hardening, flavor text pass, final deploy	All

Suggested Claude Handoff Chunks

1. **Chunk A:** M0 + M1 (foundation + music bootstrap)
2. **Chunk B:** M2 + M3 + M4 (complete music experience)
3. **Chunk C:** M5 (setup module — unblocks everything else)
4. **Chunk D:** M6 + M7 (games + news)
5. **Chunk E:** M8 (outages)
6. **Chunk F:** M9 + M10 + M11 (Peaches, full)
7. **Chunk G:** M12 + M13 (polish + ship)

14. Risks, Fallbacks & Open Questions

Risks

Risk	Likelihood	Mitigation
YouTube bot detection	Medium	youtube-source plugin, home IP advantage, optional cookies/poToken, fallback source priority
Downdetector.com Cloudflare block	Medium	DownStatus API as primary; official status pages as fallback
Overwatch 2 has no official API	Confirmed	Rely on OverFast (unofficial) + crowdsourced outage sources
DBD player stats unavailable cleanly	Confirmed	Wiki data only; skip player stats
Host machine sleeps / Docker shuts down	Medium	<code>/diagnose</code> command, startup log message, <code>restart: unless-stopped</code>
Admin token leak	Low but severe	2FA, <code>.env</code> only, rotation procedure documented
Spotify/Apple credential expiry	Low	Graceful degradation — falls back to metadata → YouTube match

Open Questions

1. **Channel layout:** confirm four channels (rat-nest fallback, game-updates, server-status, ratpack-logs) — or combine news + outages into one? *Recommendation: keep separate for cleaner notification control.*
2. **Admin vs. curated perms:** admin granted per this plan; confirm 2FA is enabled on the bot account.
3. **Project Zomboid:** does she run a dedicated server, or just play? RCON only applies if server present.
4. **Riot API key:** needed for Valorant match history; without it, only static game data (agents/maps/weapons) works.
5. **Peaches image assets:** hole images, turd image, rat portraits — will need a small curated set of placeholder images or a folder she can drop PNGs into. Spec assumes text-only with a configurable image folder path.